

P2P Project Report

Video link: https://youtu.be/8o7xPatZQYE?si=arev-A14_dFrJhcb

System Design:

This project implements a peer-to-peer network with 50+ nodes using Docker. The **bootstrap node** (bootstrap.py) runs on port 5000 and maintains a central registry of peer URLs using Flask and a thread-safe set with **threading.Lock()**. New peers automatically register via POST **/register** when they start up.

The **peer nodes** (node.py) each have a unique UUID and run Flask servers with four endpoints: / (health check), /peers (list known peers), /message (receive messages), and /broadcast (send to all peers). A background thread runs `peer_discovery_loop()` which registers with the bootstrap after 5 seconds, discovers other peers, then updates the peer list every 30 seconds.

Direct P2P Communication: After discovering peers from the bootstrap, nodes communicate directly without involving the bootstrap. The /broadcast endpoint sends HTTP POST requests directly to each peer's /message endpoint using the discovered peer URLs.

Docker Compose orchestrates 51 containers (1 bootstrap + 50 nodes) on a bridge network. The **generate_compose.py** script uses **PyYAML** to dynamically create **docker-compose-large.yml** with all 50 node definitions and proper port mappings (5001-5050).

How Peer Discovery Works:

1. Node starts and waits 5 seconds
2. Node registers itself with bootstrap: **POST http://bootstrap:5000/register**
3. Node requests peer list: **GET http://bootstrap:5000/peers**
4. Node stores peer URLs locally (excluding itself)
5. Node updates peer list every 30 seconds automatically

All peer operations use with **peers_lock**: for thread safety, preventing race conditions between the discovery thread and Flask request handlers.

Screenshots

Bootstrap showing all registered peers:

```
(venv) PS C:\Users\melod\CECS-327\Project-3-P2P> curl.exe http://localhost:5000/peers
{
  "peers": [
    "http://node4:5000",
    "http://node8:5000",
    "http://node23:5000",
    "http://node46:5000",
    "http://node26:5000",
    "http://node39:5000",
    "http://node29:5000",
    "http://node17:5000",
    "http://node10:5000",
    "http://node37:5000",
    "http://node24:5000",
    "http://node31:5000",
    "http://node5:5000",
    "http://node22:5000",
    "http://node35:5000",
    "http://node25:5000",
    "http://node2:5000",
    "http://node45:5000",
    "http://node6:5000",
    "http://node15:5000",
    "http://node7:5000",
    "http://node3:5000",
    "http://node30:5000",
    "http://node33:5000",
    "http://node43:5000",
    "http://node38:5000",
    "http://node34:5000",
    "http://node50:5000",
    "http://node20:5000",
    "http://node40:5000",
    "http://node42:5000",
    "http://node78:5000"
  ]
}
```

Individual nodes showing their peer lists:

```
(venv) PS C:\Users\melod\CECS-327\Project-3-P2P> curl.exe http://localhost:5001/peers
{
  "node_id": "19fc9836-7f10-4ff2-b524-fa3c0db375bd",
  "peers": [
    "http://node21:5000",
    "http://node28:5000",
    "http://node18:5000",
    "http://node12:5000",
    "http://node36:5000"
  ]
}
```

```
(venv) PS C:\Users\melod\CECS-327\Project-3-P2P> curl.exe http://localhost:5035/peers
{
  "node_id": "df4fbb08-e309-414f-a62c-4bca09a1f24f",
  "peers": [
    "http://node29:5000",
    "http://node27:5000",
    "http://node2:5000",
    "http://node31:5000",
    "http://node5:5000",
    "http://node10:5000",
    "http://node12:5000",
    "http://node42:5000",
    "http://node33:5000",
    "http://node46:5000",
    "http://node7:5000",
    "http://node41:5000",
    "http://node47:5000",
    "http://node15:5000",
    "http://node44:5000",
    "http://node30:5000",
    "http://node6:5000",
    "http://node37:5000",
    "http://node40:5000",
    "http://node18:5000",
    "http://node49:5000",
    "http://node22:5000",
    "http://node11:5000",
    "http://node32:5000",
    "http://node4:5000",
    "http://node3:5000",
    "http://node36:5000",
    "http://node9:5000",
    "http://node25:5000",
    "http://node13:5000",
    "http://node78:5000"
  ]
}
```

Successful broadcast response:

```

C:\Users\wlad\Documents\Project-3-P2P> curl.exe -X POST http://localhost:5000/broadcast -H 'Content-Type: application/json' -d '{"msg": "Hello gang!"}'
{"broadcast_results": [
  {
    "peer": "http://node0:5000",
    "response": 200,
    "status": "sent"
  },
  {
    "peer": "http://node1:5000",
    "response": 200,
    "status": "sent"
  },
  {
    "peer": "http://node2:5000",
    "response": 200,
    "status": "sent"
  }
]}

```

Logs showing received messages:

node14

614cd8812795

project-3-p2p-node14:latest

STATUS

Running (12 minutes ago)

5014.5000

Logs

Inspect

Bind mounts

Exec

Files

Stats

Running on <http://172.18.0.21:5000>

Press CTRL+C to quit

Restarting with stat

Starting node 18083bd5-4d30-4415-8413-56142a264b86 on port 5000

Debugger is active!

Debugger PIN: 316-056-039

[18083bd5-4d30-4415-8413-56142a264b86] Successfully registered with bootstrap

[18083bd5-4d30-4415-8413-56142a264b86] Discovered 24 peers

[18083bd5-4d30-4415-8413-56142a264b86] Discovered 49 peers

[18083bd5-4d30-4415-8413-56142a264b86] Discovered 49 peers

[18083bd5-4d30-4415-8413-56142a264b86] Received message from 19fc9836-7f10-4ff2-b524-fa3c8db375bd: Hello gang!

node2

614cd8812795

project-3-p2p-node2:latest

STATUS

Running (13 minutes ago)

5002.5000

Logs

Inspect

Bind mounts

Exec

Files

Stats

Starting node 283c9b9a-3d9e-4165-b6fb-0f8ea8467ef4 on port 5000

Serving Flask app 'node'

Debug mode: on

WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.

Running on all addresses (0.0.0.0)

Running on <http://127.0.0.1:5000>

Running on <http://172.18.0.46:5000>

Press CTRL+C to quit

Restarting with stat

Starting node 7d627f2e-0b85-4389-93bb-6103441003b6 on port 5000

Debugger is active!

Debugger PIN: 722-424-389

[7d627f2e-0b85-4389-93bb-6103441003b6] Successfully registered with bootstrap

[7d627f2e-0b85-4389-93bb-6103441003b6] Discovered 49 peers

[7d627f2e-0b85-4389-93bb-6103441003b6] Discovered 49 peers

[7d627f2e-0b85-4389-93bb-6103441003b6] Discovered 49 peers

[7d627f2e-0b85-4389-93bb-6103441003b6] Received message from 19fc9836-7f10-4ff2-b524-fa3c8db375bd: Hello gang!

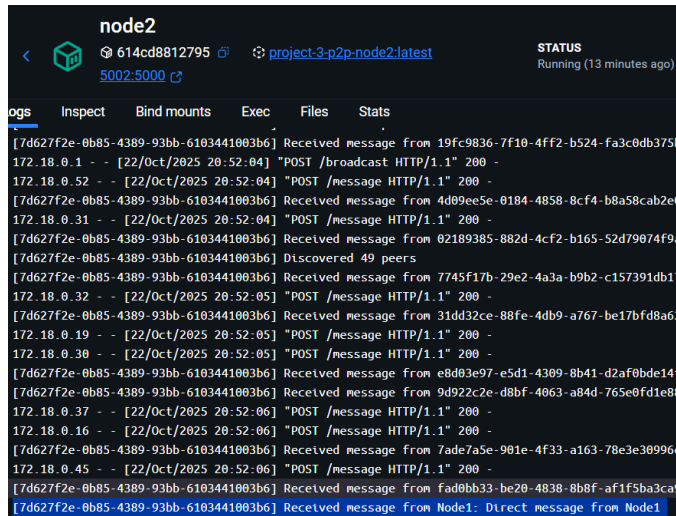
Test script output

```

# Node 3 -> Node 1
test_pairs = [
    (active_nodes[0], active_nodes[1]),
    (active_nodes[1], active_nodes[2]),
    (active_nodes[2], active_nodes[0])
]

for sender, receiver in test_pairs:
    try:
        response = requests.post(
            f'http://localhost:{5000+receiver}/message',
            json={'sender': f'Node{sender}', 'msg': f'Direct message from Node{sender}'},
            timeout=5
        )
        if response.status_code == 200:
            print(f"    ✓ Node{sender} -> Node{receiver}: Message delivered")
        else:
            print(f"    ✗ Node{sender} -> Node{receiver}: Failed")
    except Exception as e:

```



```
node2
614cd8812795 project-3-p2p-node2:latest
5002:5000
STATUS
Running (13 minutes ago)

logs Inspect Bind mounts Exec Files Stats
[7d627f2e-0b85-4389-93bb-6103441003b6] Received message from 19fc9836-7f10-4ff2-b524-fa3c0db3751
172.18.0.1 - - [22/Oct/2025 20:52:04] "POST /broadcast HTTP/1.1" 200 -
172.18.0.52 - - [22/Oct/2025 20:52:04] "POST /message HTTP/1.1" 200 -
[7d627f2e-0b85-4389-93bb-6103441003b6] Received message from 4d09ee5e-0184-4858-8cf4-b8a58cab2e
172.18.0.31 - - [22/Oct/2025 20:52:04] "POST /message HTTP/1.1" 200 -
[7d627f2e-0b85-4389-93bb-6103441003b6] Received message from 02189385-882d-4cf2-b165-52d79074f9
[7d627f2e-0b85-4389-93bb-6103441003b6] Discovered 49 peers
[7d627f2e-0b85-4389-93bb-6103441003b6] Received message from 7745f17b-29e2-4a3a-b9b2-c157391db1
172.18.0.32 - - [22/Oct/2025 20:52:05] "POST /message HTTP/1.1" 200 -
[7d627f2e-0b85-4389-93bb-6103441003b6] Received message from 31dd32ce-88fe-4db9-a767-be17bfd8a6
172.18.0.19 - - [22/Oct/2025 20:52:05] "POST /message HTTP/1.1" 200 -
172.18.0.30 - - [22/Oct/2025 20:52:05] "POST /message HTTP/1.1" 200 -
[7d627f2e-0b85-4389-93bb-6103441003b6] Received message from e8d03e97-e5d1-4309-8b41-d2af0bde14
[7d627f2e-0b85-4389-93bb-6103441003b6] Received message from 9d922c2e-d8bf-4063-a84d-765e0fd1e8
172.18.0.37 - - [22/Oct/2025 20:52:06] "POST /message HTTP/1.1" 200 -
172.18.0.16 - - [22/Oct/2025 20:52:06] "POST /message HTTP/1.1" 200 -
[7d627f2e-0b85-4389-93bb-6103441003b6] Received message from 7ade7a5e-901e-4f33-a163-78e3e30996
172.18.0.45 - - [22/Oct/2025 20:52:06] "POST /message HTTP/1.1" 200 -
[7d627f2e-0b85-4389-93bb-6103441003b6] Received message from fad0bb33-be20-4838-8b8f-af1f5ba3ca
[7d627f2e-0b85-4389-93bb-6103441003b6] Received message from Node1: Direct message from Node1
```

Challenges and Solutions:

- PowerShell JSON escaping: Used '{\"msg\": \"Hello\"}' format for curl.exe
- Container networking: Changed from localhost to Docker DNS names (<http://node1:5000>)
- Output buffering: Added flush=True to print statements for immediate logs
- Thread safety: Used threading.Lock() to prevent race conditions
- YAML typo: Fixed container-name to container_name in generate_compose.py

Individual Contributions:

Melody:

- Implemented all Python code (node.py, bootstrap.py, test_network.py, generate_compose.py)
- Tested 50-node deployment and captured screenshots
- Recorded demonstration video

Andrea:

- Created Dockerfiles and docker-compose.yml files
- Wrote README.md and project documentation

References:

Docker Documentation: <https://docs.docker.com/>

Flask Documentation: <https://flask.palletsprojects.com/>

Python Threading: <https://docs.python.org/3/library/threading.html>

CECS 327 Course Materials